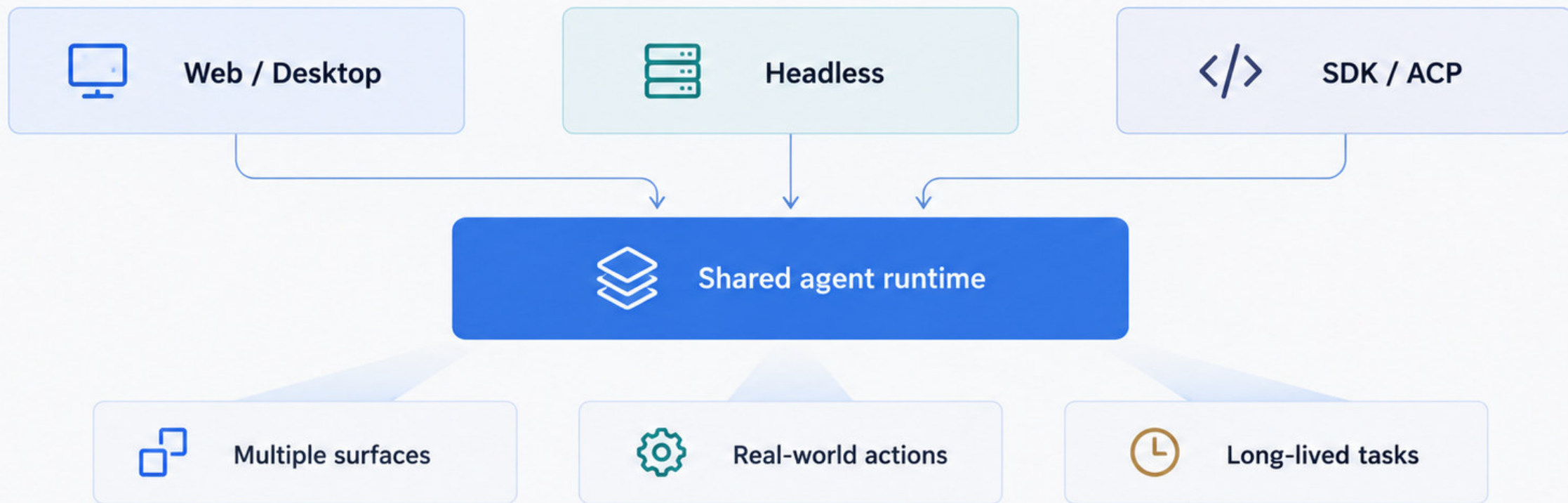


DeepSeek Harness: A Product Lens

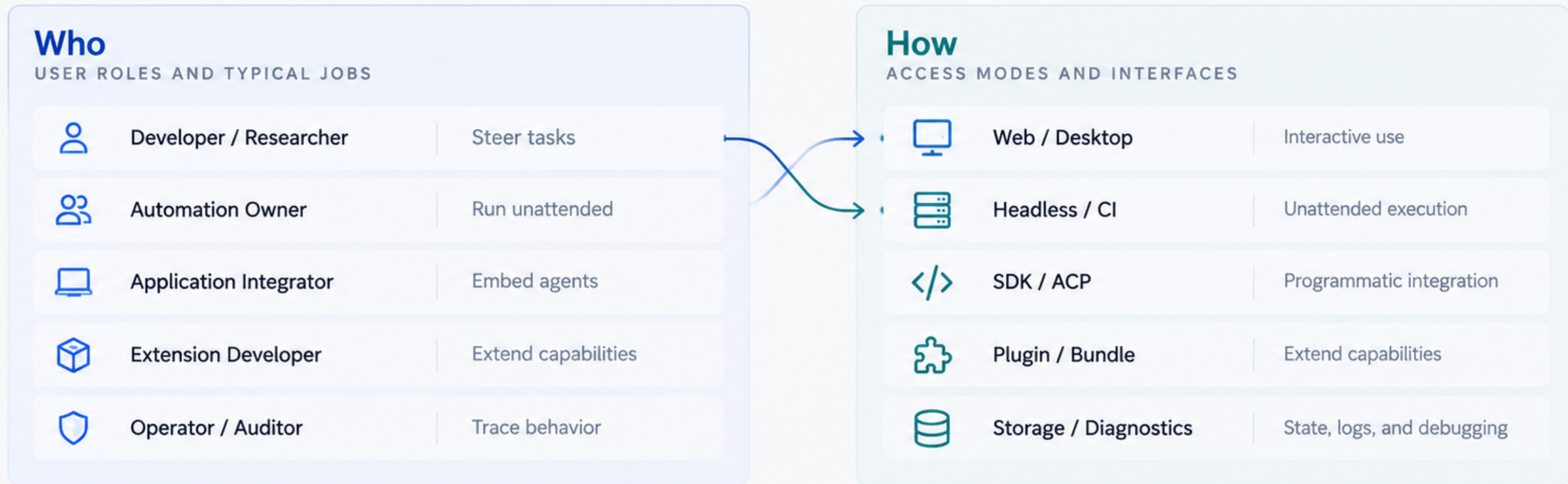
An agent product foundation, not just a coding agent.



Start with user promises, not architectural patterns.

Separate Who from How

Different roles, multiple access modes, one shared runtime.



One person can switch modes. Runtime semantics stay shared.

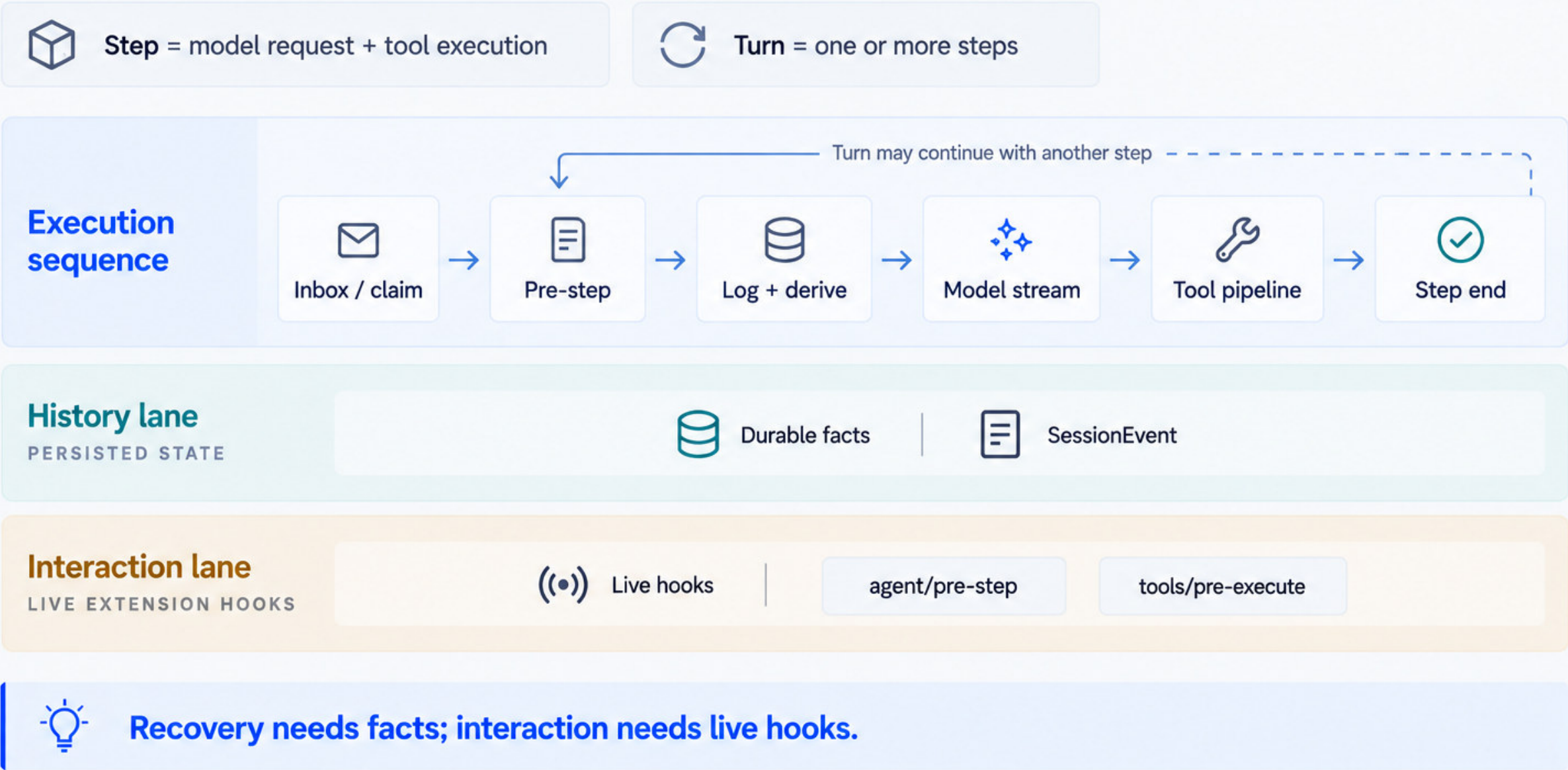
Let User Journeys Shape Architecture

From user expectations to guarantees to concrete design.



Follow user journeys, not the other way around.

Trace One Turn Before Studying Plugins



Three Composition Layers, Three Decisions

Profile



Which application surface?



Process



web / headless / sdk / acp

Bundle



Which installed capabilities?



Deployment



base + mode + user bundles

Agent Preset



What can this agent do?



Session



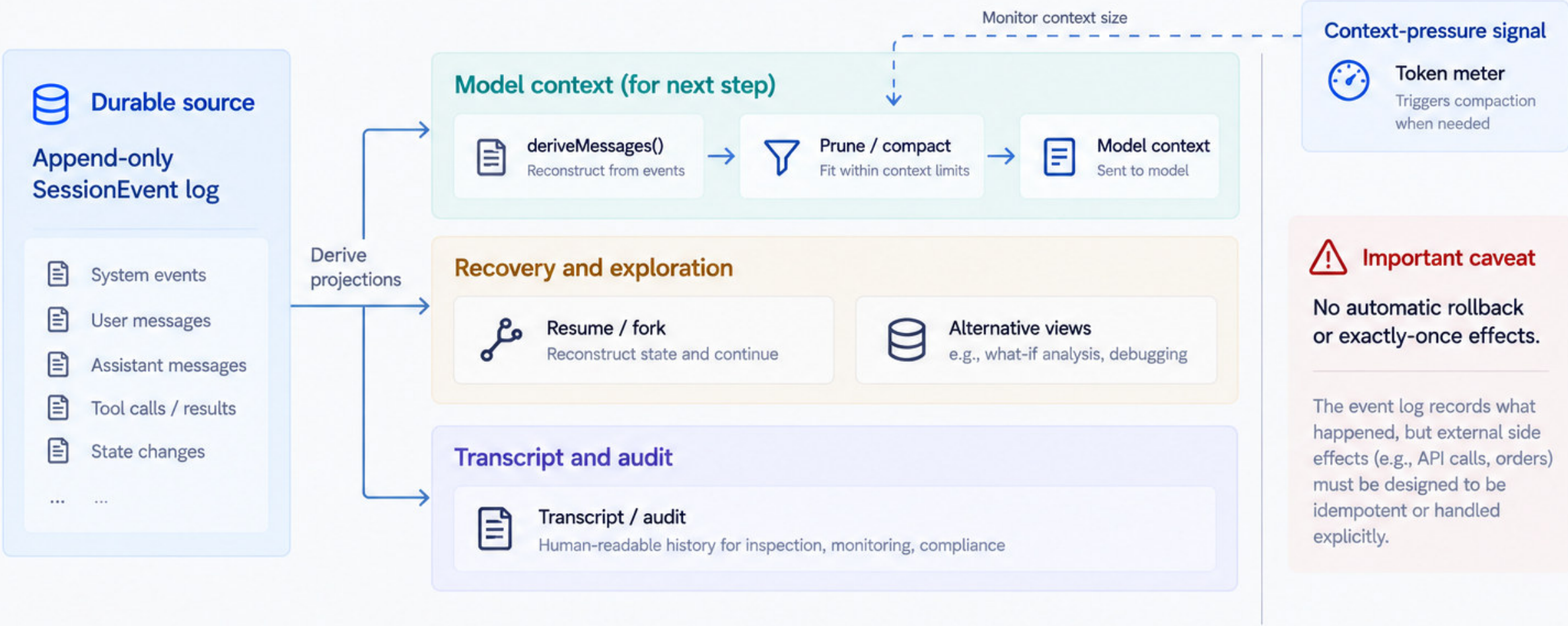
tools / prompt / compaction



Application differences are not agent differences.

Durable History Is Not Model Context

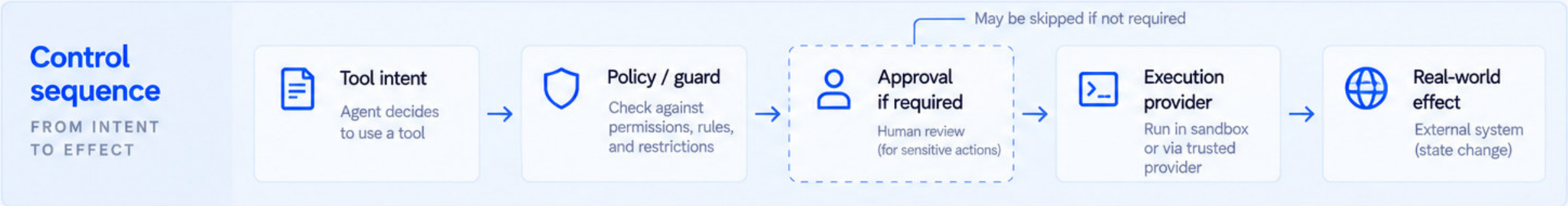
The event log preserves facts; model context and other views are derived.



Keep complete facts; reduce the model's working context.

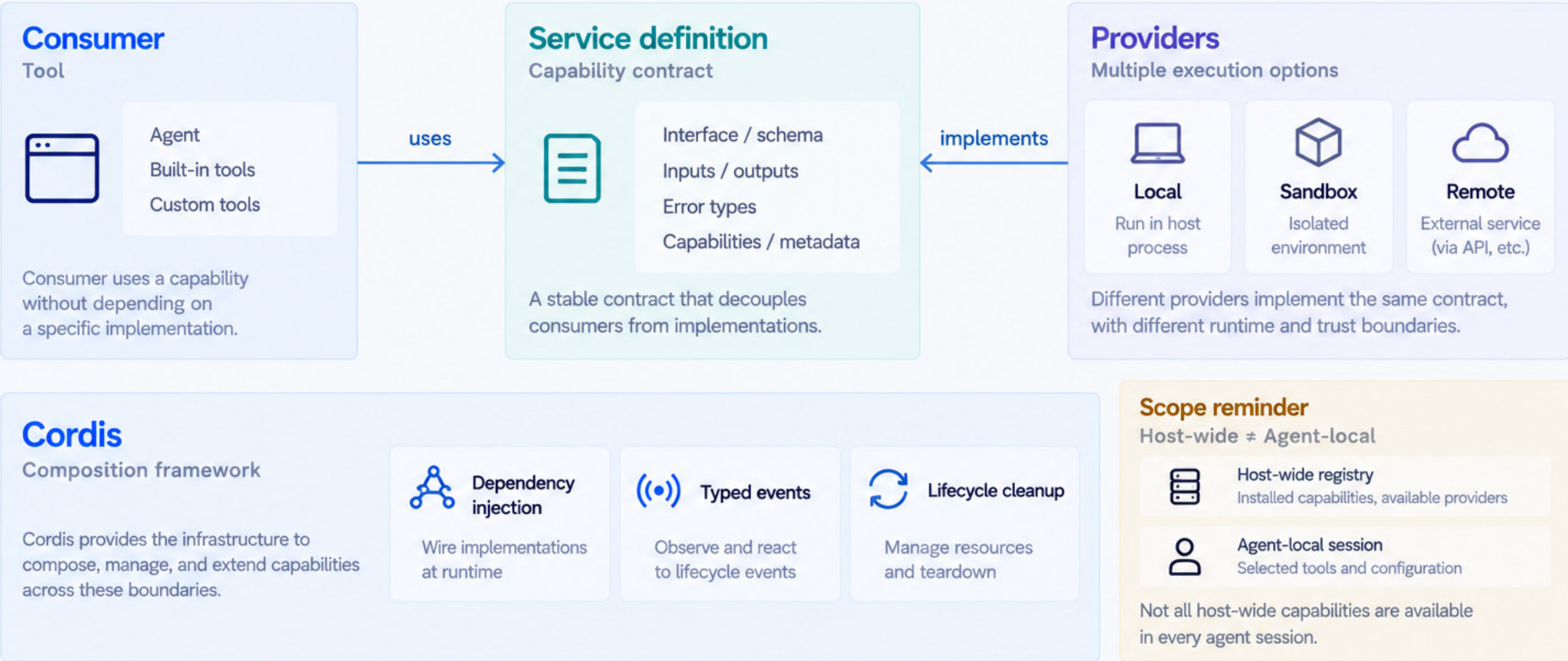
Control Side Effects Before Execution

Policy, human approval, and execution restrictions have different responsibilities.



Make Capabilities Replaceable, Not the Loop

A clear seam between consumer, service definition, and provider.



Extend through seams; keep ownership and lifecycle explicit.

Which Complexity Is Worth Paying For?

Different sources of complexity serve different purposes.

For DSH's product promises;
report assessment.



Core boundaries

JUSTIFIED BY PRODUCT PROMISES

Essential complexity to make DSH reliable, safe, and usable.



Lifecycle
Start, stop, recovery, cleanup



Durable state
Event log, reproducibility, audit



Permissions
Sandbox, approval, execution control



Runtime separation
Host, sandbox, remote providers



Context control
History, compaction, token management



Product-driven investment

ADDITIONAL VALUE AS NEEDS GROW

Complexity that enables more agent types, backends, external developers, or easier maintenance.



Presets / providers
Multiple agent types and execution backends



Plugin distribution
Easier sharing and ecosystem growth



Invariant gates
Quality, safety, and compatibility checks



Potential excess

MAY BE SIMPLIFIED FOR SMALLER PROJECTS

Granularity and abstraction that add cognitive and maintenance cost, especially if requirements are simpler.



Everything as a plugin
Higher indirection, steeper learning curve



Over-fragmented packages
More surface area, harder to maintain



Remove this abstraction: which user scenario gets worse?

If removing a layer does not clearly hurt a real user scenario, it may be unnecessary for your project.

DeepSeek Harness | Snapshot c36a83f · 2026-09-22 09 / 10

Make Architecture Legible to Coding Agents

Turn insights from DSH into a practical development method.

